Appello 11 febbraio 2015



Politecnico di Milano

Anno accademico 2013-2014

Ingegneria del Software

Cognome:		
Nome:		Matricola:
Sezione (segnarne una): ☐ Baresi	☐ Ghezzi	☐ San Pietro

Istruzioni

1. La mancata indicazione dei dati anagrafici e della sezione comporta l'annullamento del compito.
2. Al termine, consegnare solo i fogli distribuiti utilizzando il retro delle pagine in caso di necessità. Non separare questi fogli. Eventuali fogli di brutta, ecc. non verranno in nessun caso presi in considerazione. È possibile scrivere in matita.
3. È possibile consultare liberamente libri, manuali o appunti. È proibito l'uso di ogni dispositivo elettronico (quali calcolatrici tascabili, telefoni cellulari, ecc.).
4. Non è possibile lasciare l'aula conservando il tema della prova in corso.
5. Tempo a disposizione: 2h.

Esercizio 1:
Esercizio 2:
Esercizio 3:
Esercizio 4:
Esercizio 5:

Esercizio 1

Si consideri la classe `Sudoku`, che memorizza tutte le informazioni relative ad una partita al gioco omonimo. La classe fornisce i metodi: `inizia` per l'inizializzazione della griglia per una nuova partita, `riga` e `colonna`, che restituiscono un `ArrayList<Integer>` di nove elementi con i valori contenuti nella riga (o colonna) passata come parametro, `area`, che restituisce un altro `ArrayList<Integer>` di nove elementi con i valori della sotto-matrice da 3x3 elementi in cui viene divisa la scacchiera del Sudoku (le nove aree sono numerate da 0 a 8 muovendosi riga per riga da sinistra a destra: la prima in altro a sinistra è l'area 0 e l'ultima in basso a destra è l'area 8. Per semplicità si assuma che il valore restituito per le celle vuote sia 0. La classe offre anche un metodo `muovi` che prende tre parametri: `riga`, `colonna` e `valore` per aggiungere `valore` alla posizione specificata da `riga` e `colonna`. Si fornisca:

- la definizione in JML dell'invariante pubblico della classe `Sudoku`.
- la specifica completa del metodo `muovi`, considerando che non è possibile aggiungere un elemento in una posizione già occupata e che la griglia deve sempre rispettare le regole del gioco.

Si ricorda che il Sudoku impone che ogni riga e ogni colonna debbano contenere i numeri da 1 a 9 senza ripetizioni, o un loro sotto-insieme quando la griglia è incompleta. La stessa regola —ovvero numeri da 1 a 9 senza ripetizioni— deve valere anche per ognuna delle nove aree 3x3 all'interno della griglia.

Esercizio 2

Si consideri la classe `Counter` che fornisce questa interfaccia, descritta informalmenmte:

```
public class Counter
    public Counter(); //Inizializza this a zero
    public int get(); //Restituisce il valore di this
    public void incr(); //Incrementa  di 1 il valore di this
}
```

Domanda 1

1. L'operazione `get` è un osservatore puro. Che cosa significa esattamente?
2. Si fornisca una formalizzazione del costruttore e dell'operazione `incr` in termini dell'osservatore puro.
3. Quale invariante astratto è possibile definire per la classe `Counter`?

Domanda 2

Si consideri una sottoclasse di `Counter`, `Counter2`, che ridefinisce l'operazione `incr` in modo che il contatore venga incrementato di 2. La classe `Counter2` può essere considerata un sottotipo di `Counter`? Perché? Perché no?

Domanda 3

Si consideri ora la seguente sottoclasse `CounterN` della classe `Counter`:

```
public class CounterN extends Counter
    public CounterN(int n); //Inizializza this a un valore qualunque n
    public int get(); //Restituisce il valore di this
    public void incr(); //Incrementa  di 1 il valore di this
}
```

La classe `CounterN` può essere considerata un sottotipo di `Counter`? Perché? Perché no?

Domanda 4

Si consideri invece la seguente sottoclasse `CounterN1` della classe `Counter`:

```
public class CounterN1 extends Counter
    public CounterN1(); //Inizializza this a zero
    public int get(); //Restituisce il valore di this
    public void incr(int n); //Se n>0 incrementa di n il valore di this
                             //altrimenti lancia eccezione IllegalArgument
}
```

1. L'operazione `incr` può essere una ridefinizione dell'operazione `incr` definita per `Counter`?
2. L'introduzione dell'operazione `incr` in `CounterN1` consente di vedere `CounterN1` come sottotipo di `Counter`? Perché? Perché no?

Esercizio 3

Si vuole realizzare un programma che consente di inserire una serie di valori numerici e di visualizzarli in due modalità: una tabella di valori numerici e un diagramma a barre in cui i valori sono rappresentati da barre proporzionali al valore. Si vuole consentire di modificare i singoli valori numerici in tabella ed effettuare automaticamente la modifica conseguente del diagramma a barre.

Si fornisca un diagramma delle classi UML per rappresentare un programma che realizza questo problema, facendo riferimento a opportuni design pattern che risultino convenienti da adottare.

Esercizio 4

Si supponga di avere a disposizione la classe `Contatti` che contiene i contatti memorizzati dall'utente di un telefono cellulare. La classe `Contatti` offre gli osservatori `getNome`, `getCognome`, e `getNumeroTel`. Si fornisca:

- Un rep per la classe `Contatti`.
- L'implementazione di un Iteratore per scandire i contatti in ordine di memorizzazione.
- L'implementazione di un metodo `boolean sequenzaOrdinata(Iterator<Integer> i)` che, dato l'iteratore al punto precedente, verifica se la sequenza associata all'iteratore è ordinata, cioè gli elementi estratti dall'iteratore appaiono in ordine alfabetico per cognome. La soluzione deve essere **ricorsiva**, senza fare uso di alcuna istruzione di ciclo.

Esercizio 5

Si consideri il seguente frammento di programma:

```
1. while (numProdotti) <= PROD_MAX {
2.     ----
3.     if (costoProdotto) <= restaDaSpendere) {
3.     ----
4.        if (restaDaSpendere > 0) {
5.               numProdotti++;
6.        } else {
7.               ----
8.               break;----//qui codice che non cambia n e j
9.     } else {
10.    ----
11.    }
12. };
```

1. Si definiscano casi di test che coprano tutti branch (si considerino le parti tralasciate `----` come se fossero una singola macro-istruzione);
2. Se il criterio di copertura fosse quello di copertura delle istruzioni si otterrebbe lo stesso risultato? Perché sì? Perché no?.
3. Si calcoli la condizione logica che impone che il ciclo venga eseguito due volte e la seconda volta si esca con l'istruzione break.